

It is imperative to use the provided squared sheets for the simplified diagrams

An unjustified answer will be considered as false

We want to study the performance of a sort function on a superscalar implementation of the Mips architecture. The implementation is 2-pipeline called **SS2** studied in class. Two different versions of the same function are considered.

Consider the **Partition** function. This function takes as input an array of integers **src** and copies it into another array **trg**, separating the smallest elements from the largest. The **src** array is scanned and each element is compared with the first element of the array. If it is smaller than the first element, it is placed at the beginning of the **trg** array, otherwise at the end of the **trg** array.

<pre> unsigned int Partition (int *src, int *trg, unsigned int size) { unsigned int i = 0; unsigned int idx_d = 0; unsigned int idx_f = size - 1; int pivot = 0; pivot = src [0]; </pre>	<pre> for (i=size ; i!=0 ; i--) { if (src [i-1] < pivot) { trg [idx_d] = src [i-1]; idx_d ++; } else { trg [idx_f] = src [i-1]; idx_f --; } } return (idx_d); } </pre>
--	---

First, we assume that the memory system is perfect.

A first compiler has produced a first version of assembly code for the main loop of this function.

At loop entry, register R4 is assumed to contain the address of the **src** array, R5 the end address of the **src** array (the address immediately following the last cell of the array), R6 the address of the **trg** array, and R7 the address of the last cell in the **trg** array. R8 is assumed to contain the value of the **pivot**.

Version 1 :

```

_For : Lw      r9 , -4 (r5 )      ; src [i-1]
       Slt     r10, r9 , r8      ; src [i-1] < pivot ?
       Beq     r10, r0 , _Else
       Nop
       Sw      r9 , 0 (r6 )
       Addiu   r6 , r6 , 4
       J       _Endif
       Nop
_Else : Sw      r9 , 0 (r7 )
       Addiu   r7 , r7 , -4
_Endif: Addiu   r5 , r5 , -4
       Bne     r4 , r5 , _For
       Nop
    
```

We assume that the label **For** is located on an aligned address (multiple of 8) and that the instruction buffer is empty at the loop entry.

- 1- Analyze the execution of the loop using a simplified diagram on **SS2** in the case where the branch fails and in the case where the branch succeeds. How many cycles are required to execute an iteration in each case ?
- 2- Optimize the code using the *Software Pipeline* technique to avoid lost cycles as much as possible (explain your approach). Clearly indicate the number of stages and the instructions that compose each stage.

A second compiler has produced another version of the loop code. Once again, we assume that the label **For** is placed on an aligned address (multiple of 8) and that the instruction buffer is empty at the loop entry.

Version 2 :

```

_For : Lw      r9 , -4 (r5 )      ; src [i-1]
       Slt     r10, r9 , r8      ; src [i-1] < pivot ?
       Sll     r10, r10, 2
       Sw      r9 , 0 (r6 )
       Sw      r9 , 0 (r7 )
       Addiu   r7 , r7 , -4
       Addu    r7 , r7 , r10
       Addu    r6 , r6 , r10

       Addiu   r5 , r5 , -4
       Bne     r4 , r5 , _For
       Nop

```

- 3- Analyze the execution of the loop using a simplified diagram for **SS2**. How many cycles are needed to execute one iteration ?
- 4- Optimize the code using the Software Pipeline technique to avoid lost cycles as much as possible (explain your approach). Clearly indicate the number of stages and the instructions contained in each stage.

We now wish to study the influence of the memory sub-system on the performance of the two versions. In the remainder of this exercise, we will consider the original code of both versions.

We assume that the **src** array contains 256 integers. The addresses of the two arrays **src** and **trg** are aligned (multiples of 1024).

We aim at evaluating the execution performance of the two codes on different hardware platforms. On all platforms, the processor is connected to a separate data cache and instruction cache. The instruction cache is perfect (miss rate = 0).

It is assumed that at the start of function execution, the data cache is empty.

The data cache has a capacity of 1 Kbyte. It is a Direct Mapping cache, and a cache block contains 64 bytes. Cache access time is 1 cycle in the case of a hit.

The data cache is connected to the primary memory via a bus. This bus allows one transaction to be carried out at a time. Each cycle, 4 bytes (1 word) can be transferred on the bus. On average, 6 cycles are required to initialize the transfer. So, in the case of a Miss in the cache, it takes 1 cycle to detect the Miss, then 22 cycles ($6 + 1 \times 64 / 4$) to transfer the missing block. After the block is loaded, it takes another 1 cycle to update the cache and a further 1 cycle to respond to the processor.

On the first platform, we use a Write Through data cache that has a posted write buffer with 2 places. For a write, if the block to be written is not present, it is not loaded into the cache. If there is at least one free space in the buffer, a write costs 1 cycle, the time required to write into the buffer. Each posted write request is carried out by a transaction on the bus. A posted write request present in the buffer is deleted at the end of the transaction on the bus, which requires an average of 7 $(6 + 1 \times 4 / 4)$ cycles.

5- What is the advantage of having 2 places in the posted writes buffer ?

6- How many Misses occur during the execution of Version 1 ?

Calculate the average number of additional cycles per iteration due to reads.

Calculate the average number of additional cycles per iteration due to writes.

Deduce the average number of cycles per iteration if the branch succeeds and if it fails.

7- How many Misses occur during the execution of Version 2 ?

Calculate the average number of additional cycles per iteration due to reads.

Calculate the average number of additional cycles per iteration due to writes.

Deduce the average number of cycles per iteration.

On the second platform, a Write Back data cache is used. During a write, if the block to be written is not present, the cache first loads the missing block from memory and then modifies it.

Assume that all elements of the **src** array are larger than the pivot.

8- How many Misses occur during the execution of Version 1 (distinguish 2 cases) ?

Calculate the average number of additional cycles per iteration due to reads.

Calculate the average number of additional cycles per iteration due to writes.

Deduce the average number of cycles per iteration.

9- Same questions for Version 2.

Assume that all elements of the **src** array are smaller than the pivot.

10- How many Misses occur during the execution of Version 1 (distinguish 2 cases) ?

Calculate the average number of additional cycles per iteration due to reads.

Calculate the average number of additional cycles per iteration due to writes.

Deduce the average number of cycles per iteration.

11- Same questions for Version 2.